

Chapter 6

Learning

Outline

- ▶ Learning from Examples/Observation
- ▶ Knowledge in Learning
- ▶ Learning Probabilistic Models
- ▶ Neural Networks

Introduction to Learning

- ▶ Machine Learning is the study of how to build computer systems that adapt and improve with experience.
- ▶ It is a subfield of Artificial Intelligence and intersects with cognitive science, information theory, and probability theory, among others.
- ▶ Classical AI deals mainly with *deductive* reasoning, learning represents *inductive* reasoning.
- ▶ Deductive reasoning arrives at answers to queries relating to a particular situation starting from a set of general axioms, whereas inductive reasoning arrives at general axioms from a set of particular instances.
- ▶ Classical AI often suffers from the knowledge acquisition problem in real life applications where obtaining and updating the knowledge base is costly and prone to errors.
- ▶ Machine learning serves to solve the knowledge acquisition bottleneck by obtaining the result from data by induction.

Introduction to Learning ... Cont'd

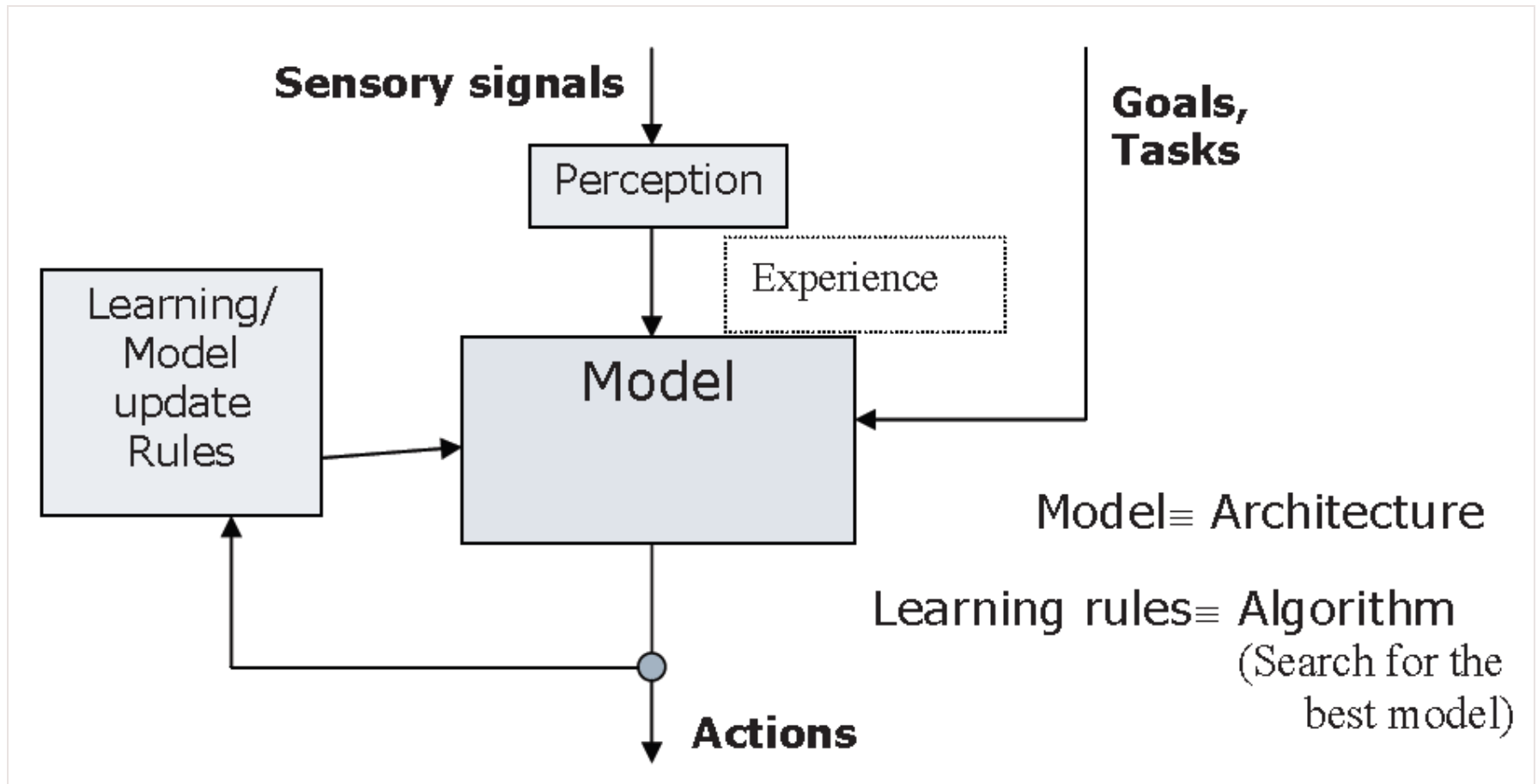
- ▶ Machine learning is particularly attractive in several real life problem because of the following reasons:
 - Some tasks cannot be defined well except by example
 - Working environment of machines may not be known at design time
 - Explicit knowledge encoding may be difficult and not available
 - Environments change over time
 - Biological systems learn
- ▶ Recently, learning is widely used in a number of application areas including,
 - Data mining and knowledge discovery
 - Speech/image/video (pattern) recognition
 - Adaptive control
 - Autonomous vehicles/robots
 - Decision support systems
 - Bioinformatics
 - WWW

Introduction to Learning ... Cont'd

- ▶ Formally, a computer program is said to **learn** from experience E with respect to some class of tasks T and performance measure P , if its performance at tasks in T , as measured by P , improves with experience E .
- ▶ Thus a learning system is characterized by:
 - task T
 - experience E , and
 - performance measure P
- ▶ **Examples:**
- ▶ *Learning to play chess*
 - ▶ T: Play chess
 - ▶ P: Percentage of games won in world tournament
 - ▶ E: Opportunity to play against self or other players
- ▶ *Learning to drive a van*
 - ▶ T: Drive on a public highway using vision sensors
 - ▶ P: Average distance traveled before an error (according to human observer)
 - ▶ E: Sequence of images and steering actions recorded during human driving.

Introduction to Learning ... Cont'd

The block diagram of a generic learning system which can realize the above definition is shown below:



Introduction to Learning ... Cont'd

- ▶ As can be seen from the above diagram the system consists of the following components:
- ▶ • *Goal*: Defined with respect to the task to be performed by the system
- ▶ • *Model*: A mathematical function which maps perception to actions
- ▶ • *Learning rules*: Which update the model parameters with new experience such that the performance measures with respect to the goals is optimized
- ▶ • *Experience*: A set of perception (and possibly the corresponding actions)

Taxonomy of Learning Systems

- ▶ Several classification of learning systems are possible based on the above components as follows:
- ▶ **Goal/Task/Target Function:**
- ▶ *Prediction*: To predict the desired output for a given input based on previous input/output pairs. E.g., to predict the value of a stock given other inputs like market index, interest rates etc.
- ▶ *Categorization*: To classify an object into one of several categories based on features of the object. E.g., a robotic vision system to categorize a machine part into one of the categories, spanner, hammer etc based on the parts' dimension and shape.
- ▶ *Clustering*: To organize a group of objects into homogeneous segments. E.g., a satellite image analysis system which groups land areas into forest, urban and water body, for better utilization of natural resources.
- ▶ *Planning*: To generate an optimal sequence of actions to solve a particular problem. E.g., an Unmanned Air Vehicle which plans its path to obtain a set of pictures and avoid enemy anti-aircraft guns.

Taxonomy of Learning Systems ... Cont'd

- ▶ Models:
 - ▶ Propositional and FOL rules
 - ▶ Decision trees
 - ▶ Linear separators
 - ▶ Neural networks
 - ▶ Graphical models
 - ▶ Temporal models like hidden Markov models
- ▶ **Learning Rules:**
 - ▶ Learning rules are often tied up with the model of learning used.
 - ▶ Some common rules are gradient descent, least square error, expectation maximization and margin maximization.

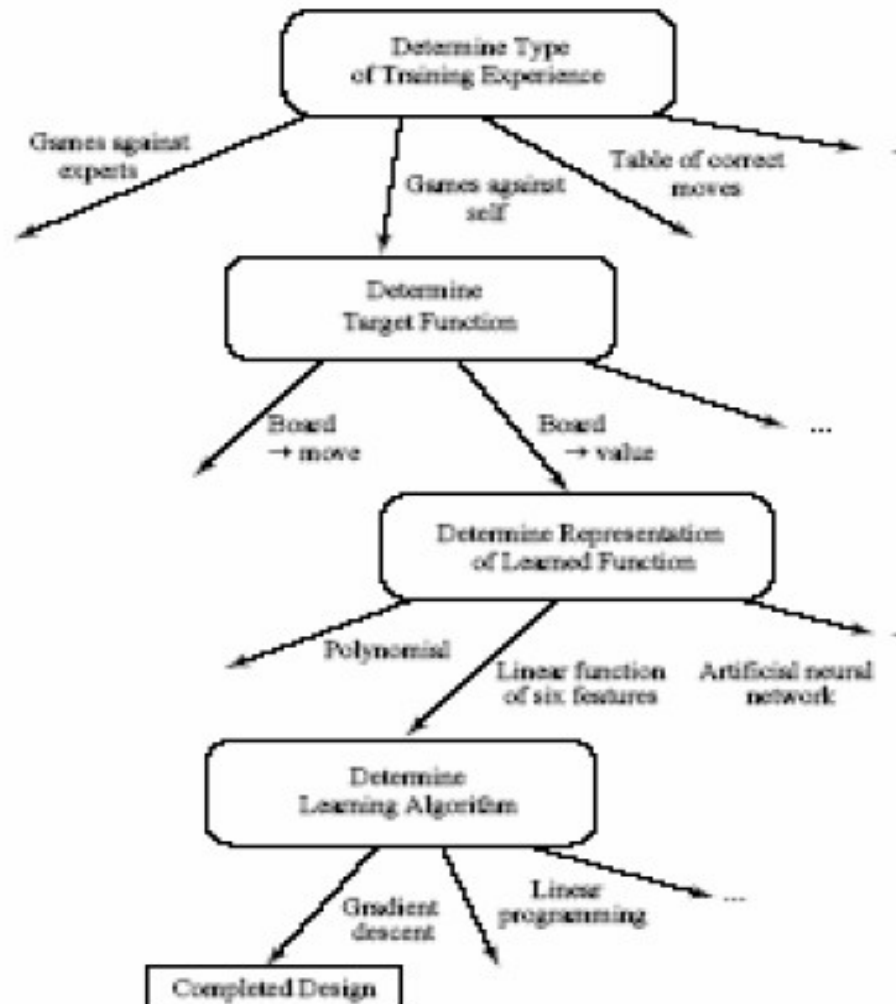
Taxonomy of Learning Systems ... Cont'd

- ▶ **Experiences:**
- ▶ Learning algorithms use experiences in the form of perceptions or perception action pairs to improve their performance. The nature of experiences available varies with applications. Some common situations are described below.
- ▶ *Supervised learning:* In supervised learning a teacher or oracle is available which provides the desired action corresponding to a perception. A set of perception action pair provides what is called a training set. Examples include an automated vehicle where a set of vision inputs and the corresponding steering actions are available to the learner.
- ▶ *Unsupervised learning:* In unsupervised learning no teacher is available. The learner only discovers persistent patterns in the data consisting of a collection of perceptions. This is also called exploratory learning. Finding out malicious network attacks from a sequence of anomalous data packets is an example of unsupervised learning.

Taxonomy of Learning Systems ... Cont'd

- ▶ *Active learning*: Here not only a teacher is available, the learner has the freedom to ask the teacher for suitable perception-action example pairs which will help the learner to improve its performance. Consider a news recommender system which tries to learn an users preferences and categorize news articles as interesting or uninteresting to the user. The system may present a particular article (of which it is not sure) to the user and ask whether it is interesting or not.
 - ▶ *Reinforcement learning*: In reinforcement learning a teacher is available, but the teacher instead of directly providing the desired action corresponding to a perception, return reward and punishment to the learner for its action corresponding to a perception. Examples include a robot in a unknown terrain where its get a punishment when its hits an obstacle and reward when it moves smoothly.
 - ▶ In order to design a learning system the designer has to make the following choices based on the application.
-

Taxonomy of Learning Systems ... Cont'd



Mathematical formulation of the inductive learning problem

- ▶ Extrapolate from a given set of examples so that we can make accurate predictions about future examples.
- ▶ **Supervised versus Unsupervised learning** Want to learn an unknown function $f(\mathbf{x}) = \mathbf{y}$, where $\mathbf{x}$ is an input example and $\mathbf{y}$ is the desired output. Supervised learning implies we are given a set of $(\mathbf{x}, \mathbf{y})$ pairs by a "teacher." Unsupervised learning means we are only given the $\mathbf{x}$ s.
- ▶ In either case, the goal is to estimate f .

Mathematical formulation of the inductive learning problem ... Cont'd

- ▶ **Inductive Bias**
- ▶ Inductive learning is an inherently conjectural process because any knowledge created by generalization from specific facts cannot be proven true; it can only be proven false. Hence, inductive inference is **falsity preserving**, not truth preserving.
- ▶ To generalize beyond the specific training examples, we need constraints or **biases** on what f is best. That is, learning can be viewed as searching the **Hypothesis Space** H of possible f functions.
- ▶ A bias allows us to choose one f over another one
- ▶ A completely unbiased inductive algorithm could only memorize the training examples and could not say anything more about other unseen examples.
- ▶ Two types of biases are commonly used in machine learning:
 - ▶ **Restricted Hypothesis Space Bias** Allow only certain types of f functions, not arbitrary ones

Mathematical formulation of the inductive learning problem ... Cont'd

- ▶ **Preference Bias** Define a metric for comparing f s so as to determine whether one is better than another
- ▶ Inductive Learning Framework
- ▶ Raw input data from sensors are preprocessed to obtain a **feature vector**, $\mathbf{x}$, that adequately describes all of the relevant features for classifying examples.
- ▶ Each $\mathbf{x}$ is a list of (attribute, value) pairs. For example,
- ▶ $\mathbf{x} = (\text{Person} = \text{Sue}, \text{Eye-Color} = \text{Brown}, \text{Age} = \text{Young}, \text{Sex} = \text{Female})$
- ▶ The number of attributes (also called features) is fixed (positive, finite). Each attribute has a fixed, finite number of possible values.
- ▶ Each example can be interpreted as a *point* in an n -dimensional **feature space**, where n is the number of attributes.

Learning From Observations

- ▶ **Concept Learning**
- ▶ **Definition:**
- ▶ The problem is to learn a function mapping examples into two classes: positive and negative. We are given a database of examples already classified as positive or negative. Concept learning: the process of inducing a function mapping input examples into a Boolean output.
- ▶ Examples:
 - ▶ Classifying objects in astronomical images as stars or galaxies
 - ▶ Classifying animals as vertebrates or invertebrates
- ▶ Example: Classifying Mushrooms
- ▶ Class of Tasks: Predicting poisonous mushrooms
- ▶ Performance: Accuracy of classification
- ▶ Experience: Database describing mushrooms with their class
- ▶ Knowledge to learn: Function mapping mushrooms to $\{0,1\}$ where 0: not-poisonous and 1:poisonous

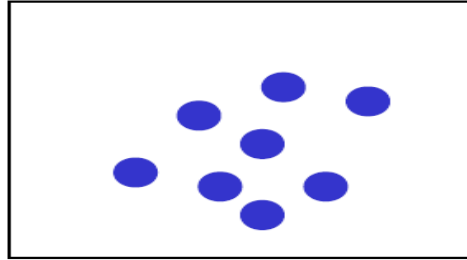
Learning From Observations ... Cont'd

- ▶ Representation of target knowledge: conjunction of attribute values.
- ▶ Learning mechanism: candidate-elimination
- ▶ *Representation of instances:*
- ▶ Features:
 - ▶ color {red, brown, gray}
 - ▶ size {small, large}
 - ▶ shape {round, elongated}
 - ▶ land {humid, dry}
 - ▶ air humidity {low, high}
 - ▶ texture {smooth, rough}
- ▶ *Input and Output Spaces:*
 - ▶ X : The space of all possible examples (input space).
 - ▶ Y: The space of classes (output space).
- ▶ An example in X is a feature vector X.
 - ▶ For instance: $X = (\text{red}, \text{small}, \text{elongated}, \text{humid}, \text{low}, \text{rough})$
 - ▶ X is the cross product of all feature values.
- ▶ Only a small subset of instances is available in the database of examples.



Learning From Observations ... Cont'd

$\mathcal{X}$



$\mathcal{Y} = \{0, 1\}$

Training Examples:

$\mathcal{D}$: The set of training examples.

$\mathcal{D}$ is a set of pairs $\{ (x, c(x)) \}$, where c is the target concept. c is a subset of the universe of discourse or the set of all possible instances.

Example of $\mathcal{D}$:

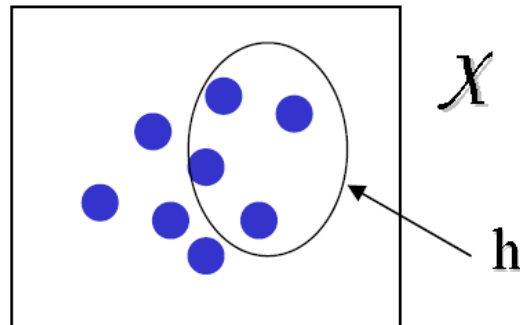
- ((red,small,round,humid,low,smooth), poisonous)
- ((red,small,elongated,humid,low,smooth), poisonous)
- ((gray,large,elongated,humid,low,rough), not-poisonous)
- ((red,small,elongated,humid,high,rough), poisonous)

Learning From Observations ... Cont'd

- ▶ **Hypothesis Representation**
- ▶ Any hypothesis h is a function from X to Y
- ▶ $h: X \rightarrow Y$
- ▶ We will explore the space of conjunctions.
- ▶ Special symbols:
 - ▶? Any value is acceptable
 - ▶0 no value is acceptable
- ▶ Consider the following hypotheses:
 - ▶(?,?,?, ?, ?, ?): all mushrooms are poisonous
 - ▶(0,0,0,0,0,0): no mushroom is poisonous

Learning From Observations ... Cont'd

- ▶ **Hypotheses Space:**
- ▶ The space of all hypotheses is represented by H
- ▶ Let h be a hypothesis in H .
- ▶ Let X be an example of a mushroom.
- ▶ if $h(X) = 1$ then X is poisonous, otherwise X is not-poisonous
- ▶ Our goal is to find the hypothesis, h^* , that is very “close” to target concept c .
- ▶ A hypothesis is said to “cover” those examples it classifies as positive.

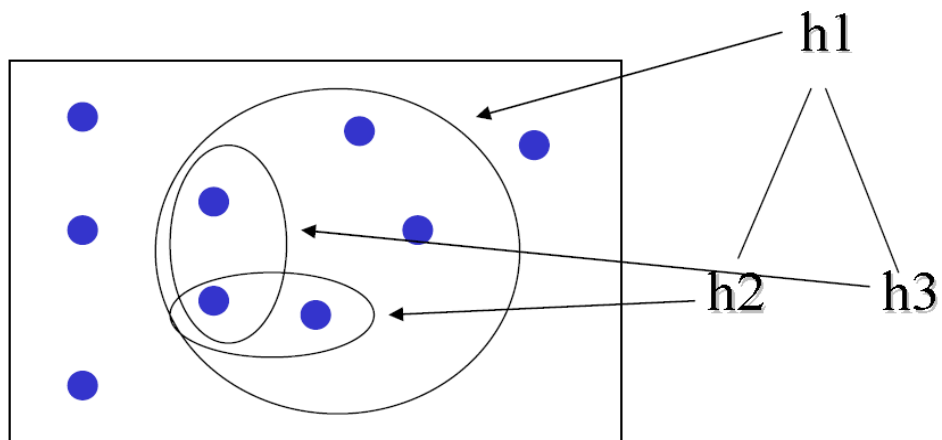


Learning From Observations ... Cont'd

- ▶ *Assumptions:*
- ▶ We will explore the space of all conjunctions.
- ▶ We assume the target concept falls within this space.
- ▶ A hypothesis close to target concept c obtained after seeing many training examples will result in high accuracy on the set of unobserved examples.
(Inductive Learning Hypothesis)

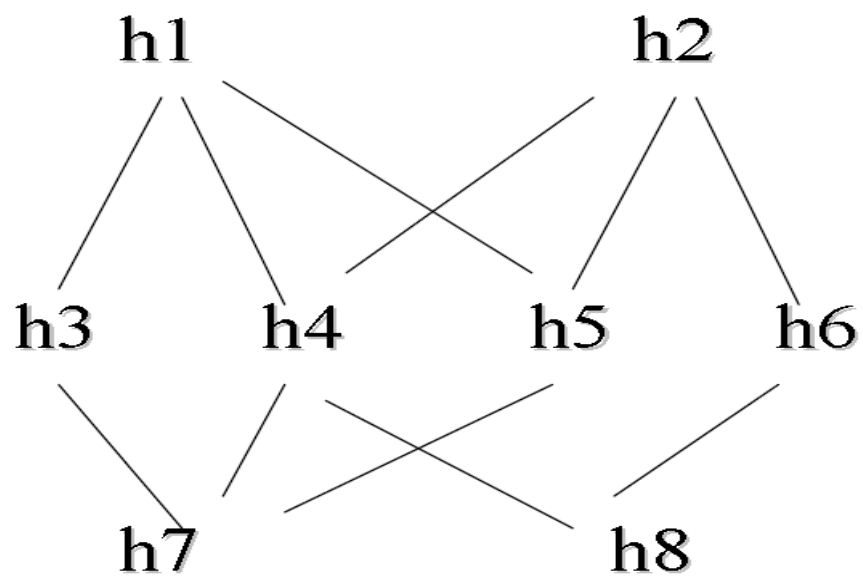
Concept Learning as Search

- ▶ We will see how the problem of concept learning can be posed as a search problem.
- ▶ We will illustrate that there is a general to specific ordering inherent to any hypothesis space.
- ▶ Consider these two hypotheses:
 - ▶ $h1 = (\text{red}, ?, ?, \text{humid}, ?, ?)$
 - ▶ $h2 = (\text{red}, ?, ?, ?, ?, ?)$
- ▶ We say $h2$ is more general than $h1$ because $h2$ classifies more instances than $h1$ and $h1$ is covered by $h2$.
- ▶ For example, consider the following hypotheses



Concept Learning as Search ... Cont'd

- ▶ h_1 is more general than h_2 and h_3 .
- ▶ h_2 and h_3 are neither more specific nor more general than each other.
- ▶ *Definitions:*
- ▶ Let h_j and h_k be two hypotheses mapping examples into $\{0,1\}$.
- ▶ We say h_j is more general than h_k iff
- ▶ For all examples X , $h_k(X) = 1 \rightarrow h_j(X) = 1$
- ▶ We represent this fact as $h_j \geq h_k$
- ▶ The $\geq$ relation imposes a partial ordering over the hypothesis space H (reflexive, antisymmetric, and transitive).
- ▶ Any input space X defines then a lattice of hypotheses ordered according to the general-specific relation:



Algorithm to Find a Maximally-Specific Hypothesis

- ▶ Algorithm to search the space of conjunctions:
 - ▶ Start with the most specific hypothesis
 - ▶ Generalize the hypothesis when it fails to cover a positive example
- ▶ *Algorithm:*
 1. Initialize **h** to the most specific hypothesis
 2. For each positive training example **X**
 - ▶ For each value **a** in **h**
 - ▶ If example **X** and **h** agree on **a**, do nothing
 - ▶ Else generalize **a** by the next more general constraint
 3. Output hypothesis **h**

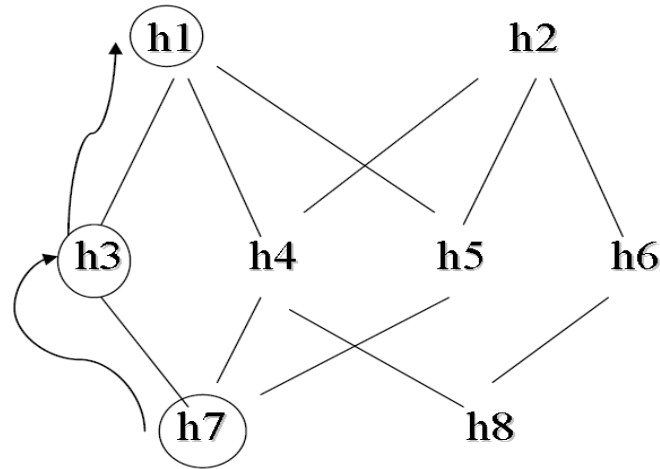
Algorithm to Find a Maximally-Specific Hypothesis

- ▶ *Example:*
- ▶ Let's run the learning algorithm above with the following examples:
- ▶ ((red,small,round,humid,low,smooth), poisonous)
- ▶ ((red,small,elongated,humid,low,smooth), poisonous)
- ▶ ((gray,large,elongated,humid,low,rough), not-poisonous)
- ▶ ((red,small,elongated,humid,high,rough), poisonous)
- ▶ We start with the most specific hypothesis: $\mathbf{h} = (0,0,0,0,0,0)$

Algorithm to Find a Maximally-Specific Hypothesis

- ▶ The first example comes and since the example is positive and **h** fails to cover it, we simply generalize **h** to cover exactly this example:
- ▶ **h** = (red,small,round,humid,low,smooth)
- ▶ Hypothesis **h** basically says that the first example is the only positive example, all other examples are negative.
- ▶ Then comes examples 2: ((red,small,elongated,humid,low,smooth), poisonous)
- ▶ This example is positive. All attributes match hypothesis **h** except for attribute shape: it has the value *elongated*, not *round*. We generalize this attribute using symbol ? yielding:
- ▶ **h**: (red,small,?,humid,low,smooth)
- ▶ The third example is negative and so we just ignore it.
- ▶ Why is it we don't need to be concerned with negative examples?
- ▶ Upon observing the 4th example, hypothesis **h** is generalized to the following:
- ▶ **h** = (red,small,?,humid,?,?)
- ▶ **h** is interpreted as any mushroom that is red, small and found on humid land should be classified as poisonous.

Algorithm to Find a Maximally-Specific Hypothesis



Algorithm to Find a Maximally-Specific Hypothesis

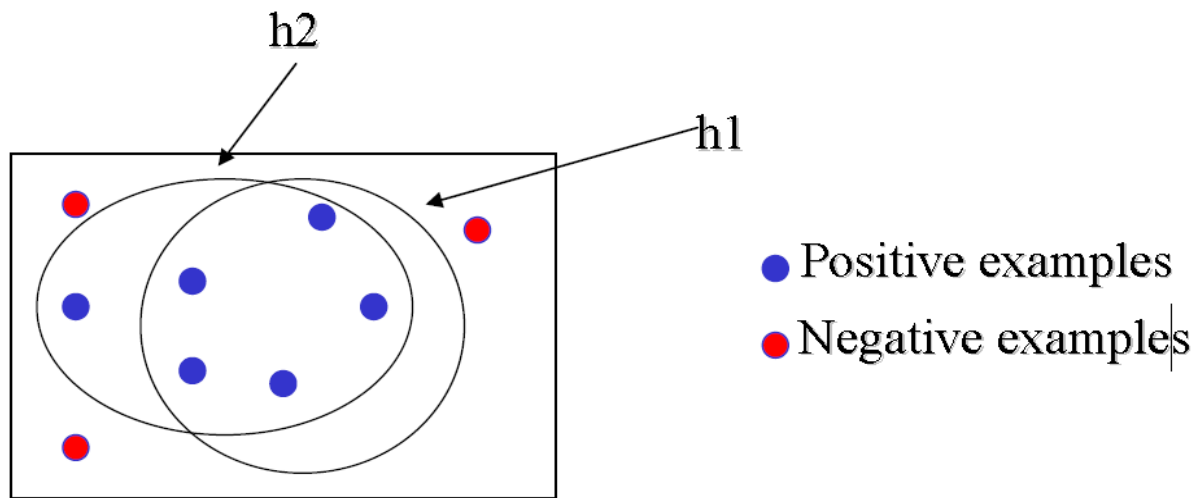
- ▶ The algorithm is guaranteed to find the hypothesis that is most specific and consistent with the set of training examples. It takes advantage of the general-specific ordering to move on the corresponding lattice searching for the next most specific hypothesis.
- ▶ Note that:
- ▶ There are many hypotheses consistent with the training data D . Why should we prefer the most specific hypothesis?
- ▶ What would happen if the examples are not consistent? What would happen if they have errors, noise?
- ▶ What if there is a hypothesis space H where one can find more than one maximally specific hypothesis h ? The search over the lattice must then be different to allow for this possibility.

Algorithm to Find a Maximally-Specific Hypothesis

- ▶ The algorithm that finds the maximally specific hypothesis is limited in that it only finds one of many hypotheses consistent with the training data.
- ▶ • The Candidate Elimination Algorithm (CEA) finds ALL hypotheses consistent with the training data.
- ▶ • CEA does that without explicitly enumerating all consistent hypotheses.
- ▶ • Applications:
 - ▶ o Chemical Mass Spectroscopy
 - ▶ o Control Rules for Heuristic Search

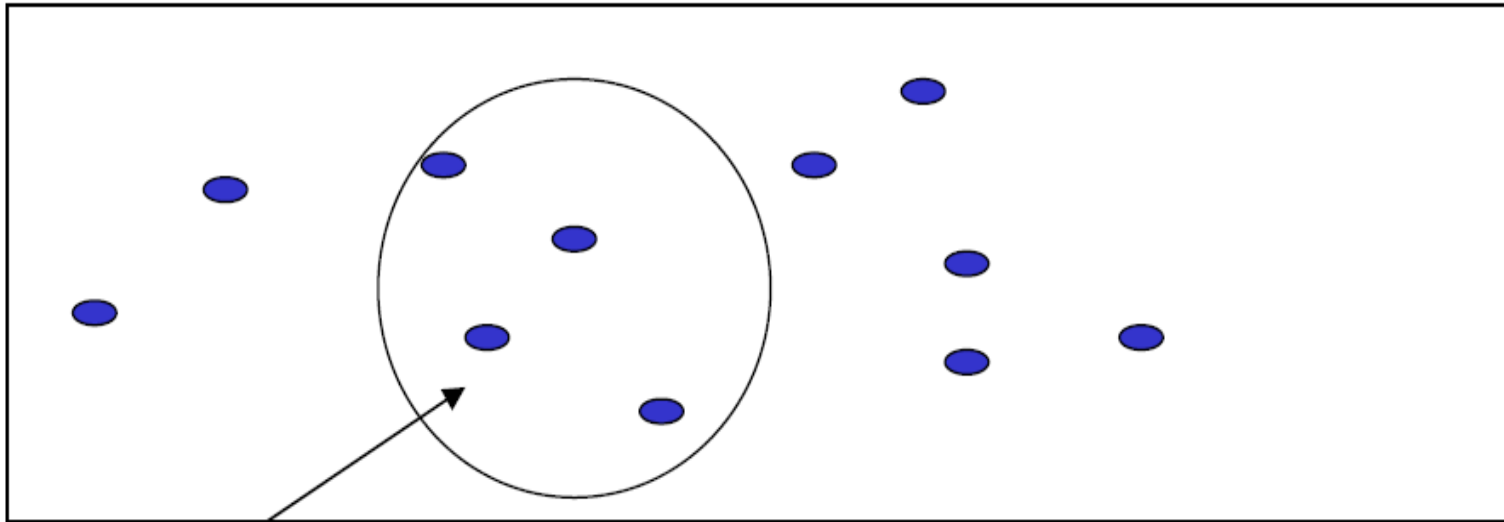
Candidate Elimination Algorithm

- ▶ *Consistency vs Coverage*
- ▶ In the following example, h_1 covers a different set of examples than h_2 , h_2 is consistent with training set D , h_1 is not consistent with training set D



Version Space

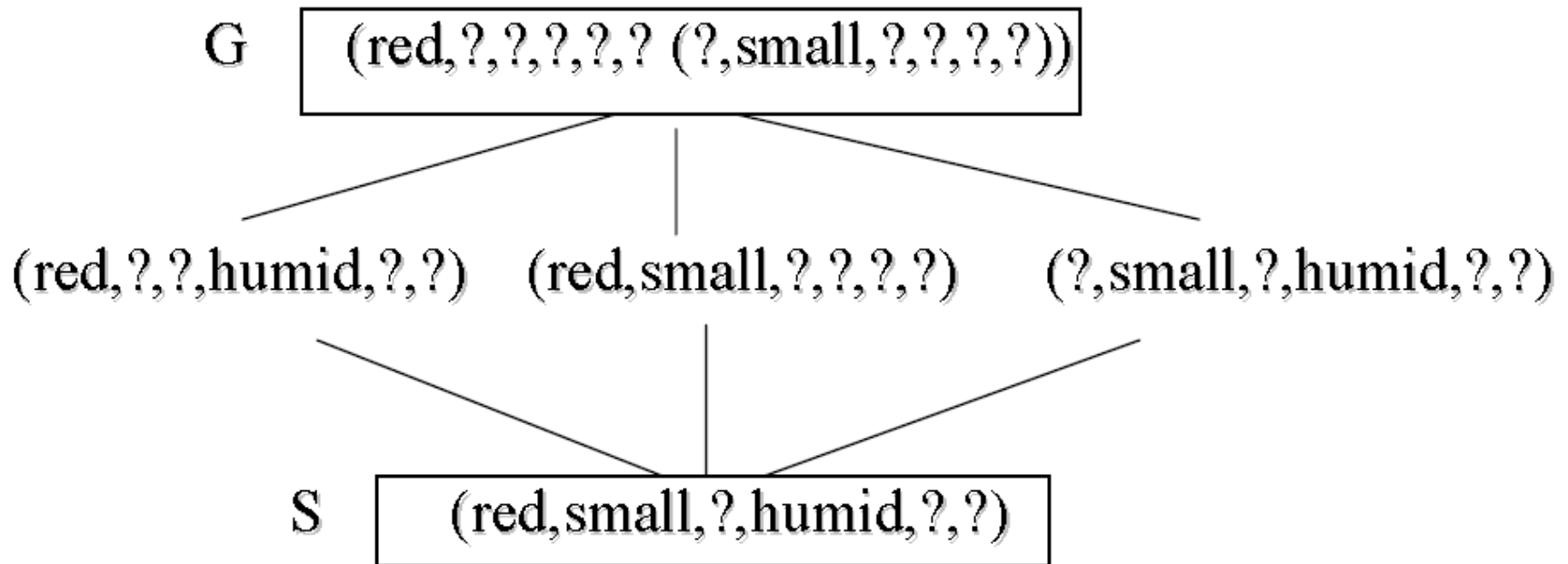
► Hypothesis Space



Version space: Subset of hypothesis from $\mathcal{H}$ consistent with training set $\mathcal{D}$.

Version Space ... Cont'd

- ▶ The version space for the mushroom example is as follows:



S: Most specific

G: Most general

The candidate elimination algorithm generates the entire version space.

The Candidate-Elimination Algorithm

- ▶ The candidate elimination algorithm keeps two lists of hypotheses consistent with the training data: (i) The list of most specific hypotheses **S** and, (ii) The list of most general hypotheses **G**. This is enough to derive the whole version space **VS**.
- ▶ Steps:
 - ▶ 1. Initialize **G** to the set of maximally general hypotheses in **H**
 - ▶ 2. Initialize **S** to the set of maximally specific hypotheses in **H**
 - ▶ 3. For each training example **X** do
 - ▶ a) If **X** is positive: generalize **S** if necessary
 - ▶ b) If **X** is negative: specialize **G** if necessary
 - ▶ 4. Output **{G,S}**
- ▶ Step (a) Positive examples

The Candidate-Elimination Algorithm

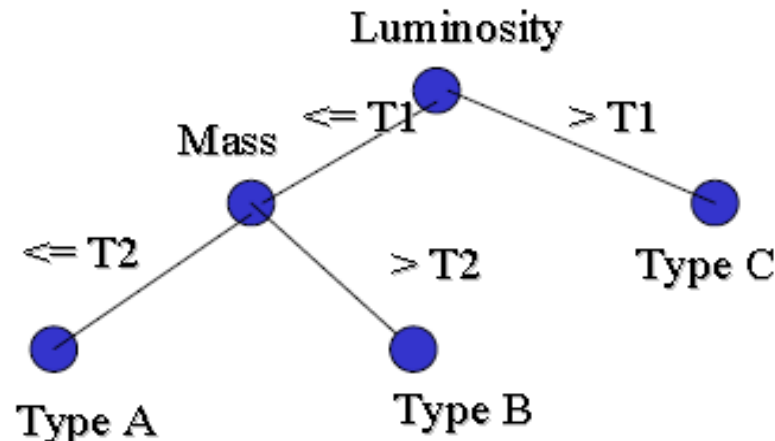
- ▶ If X is positive:
 - ▶ Remove from G any hypothesis inconsistent with X
 - ▶ For each hypothesis h in S not consistent with X
 - ▶ Remove h from S
 - ▶ Add all minimal generalizations of h consistent with X such that some member of G is more general than h
 - ▶ Remove from S any hypothesis more general than any other hypothesis in S
- ▶ Step (b) Negative examples
- ▶ If X is negative:
 - ▶ Remove from S any hypothesis inconsistent with X
 - ▶ For each hypothesis h in G not consistent with X
 - ▶ Remove g from G
 - ▶ Add all minimal generalizations of h consistent with X such that some member of S is more specific than h
 - ▶ Remove from G any hypothesis less general than any other hypothesis in G

The Candidate-Elimination Algorithm

- ▶ The candidate elimination algorithm is guaranteed to converge to the right hypothesis provided the following:
 - ▶ a) No errors exist in the examples
 - ▶ b) The target concept is included in the hypothesis space **H**
- ▶ If there exists errors in the examples:
 - ▶ a) The right hypothesis would be inconsistent and thus eliminated.
 - ▶ b) If the **S** and **G** sets converge to an empty space we have evidence that the true concept lies outside space **H**.

Rule Induction and Decision Tree - I

- ▶ **Decision Trees**
- ▶ Decision trees are a class of learning models that are more robust to noise as well as more powerful as compared to concept learning.
- ▶ Consider the problem of classifying a star based on some astronomical measurements.
- ▶ It can naturally be represented by the following set of decisions on each measurement arranged in a tree like fashion



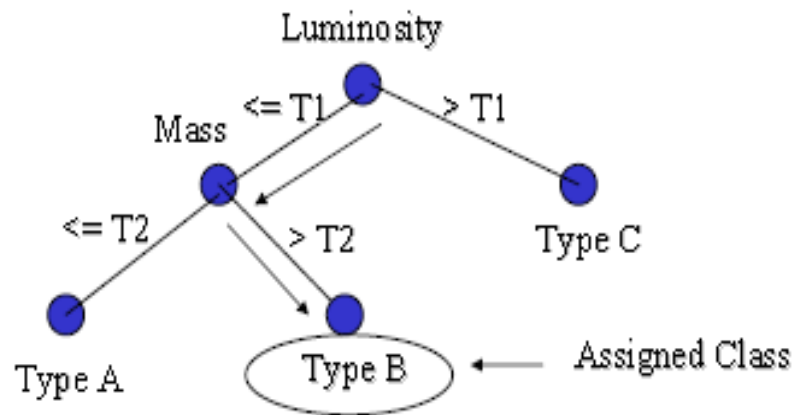
Decision Tree: Definition

- ▶ A decision-tree learning algorithm approximates a target concept using a tree representation, where each internal node corresponds to an attribute, and every terminal node corresponds to a class.
- ▶ There are two types of nodes:
 - ▶ Internal node.- Splits into different branches according to the different values the corresponding attribute can take. Example: luminosity $\leq$ T1 or luminosity $>$ T1.
 - ▶ Terminal Node.- Decides the class assigned to the example.

Classifying Examples Using Decision Tree

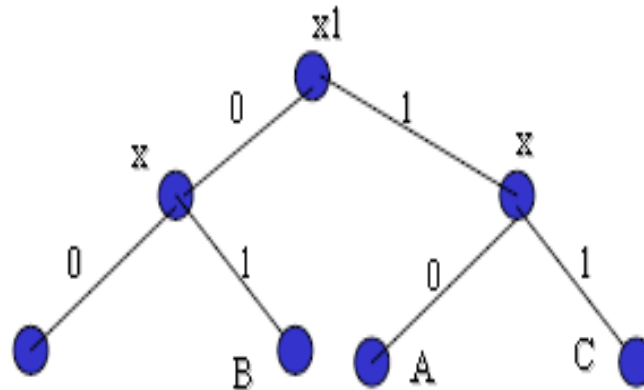
- ▶ To classify an example X we start at the root of the tree, and check the value of that attribute on X .
- ▶ We follow the branch corresponding to that value and jump to the next node.
- ▶ We continue until we reach a terminal node and take that class as our best prediction.

$X = (\text{Luminosity} \leq T1, \text{Mass} > T2)$



Cont'd...

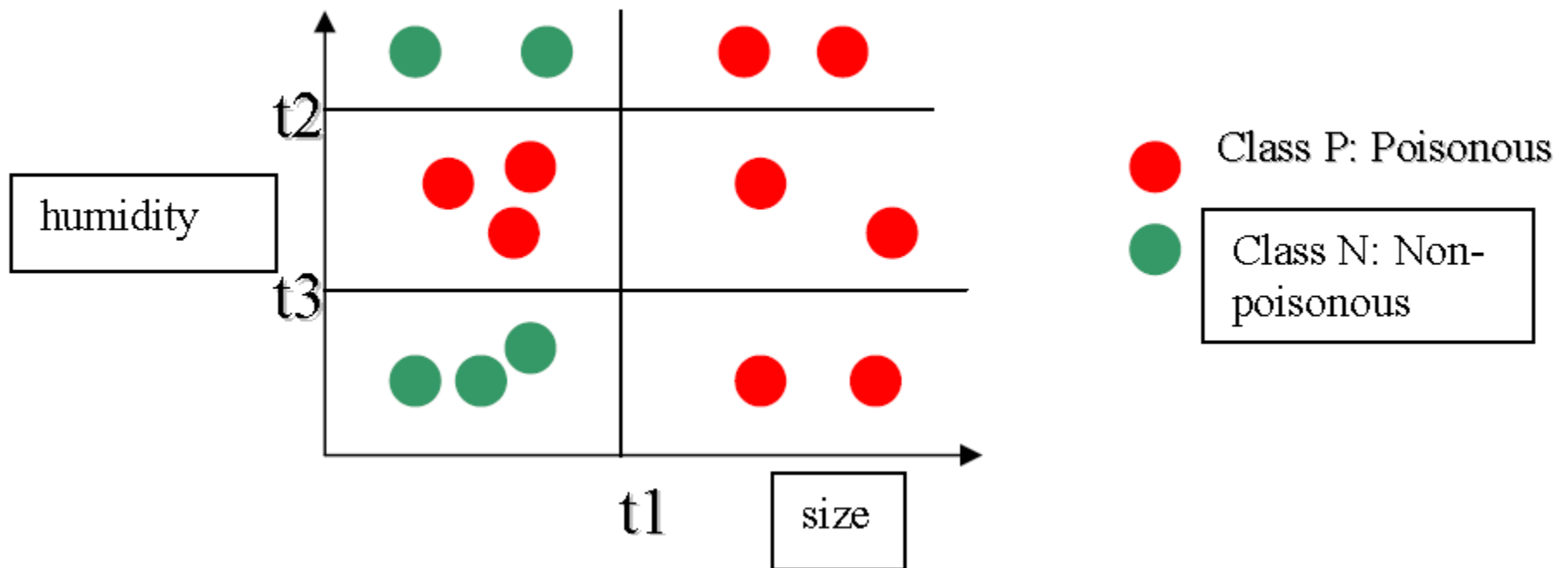
- ▶ Decision trees adopt a DNF (Disjunctive Normal Form) representation.
- ▶ For a fixed class, every branch from the root of the tree to a terminal node with that class is a conjunction of attribute values; different branches ending in that class form a disjunction.
- ▶ In the following example, the rules for class A are: $(\sim X_1 \ \& \ \sim x_2)$ OR $(X_1 \ \& \ \sim x_3)$



Decision Tree Construction

- ▶ There are different ways to construct trees from data.
- ▶ We will concentrate on the top-down, greedy search approach:
- ▶ Basic idea:
 - ▶ 1. Choose the best attribute a^* to place at the root of the tree.
 - ▶ 2. Separate training set D into subsets $\{D_1, D_2, \dots, D_k\}$ where each subset D_i contains examples having the same value for a^*
 - ▶ 3. Recursively apply the algorithm on each new subset until examples have the same class or there are few of them.

Cont'd



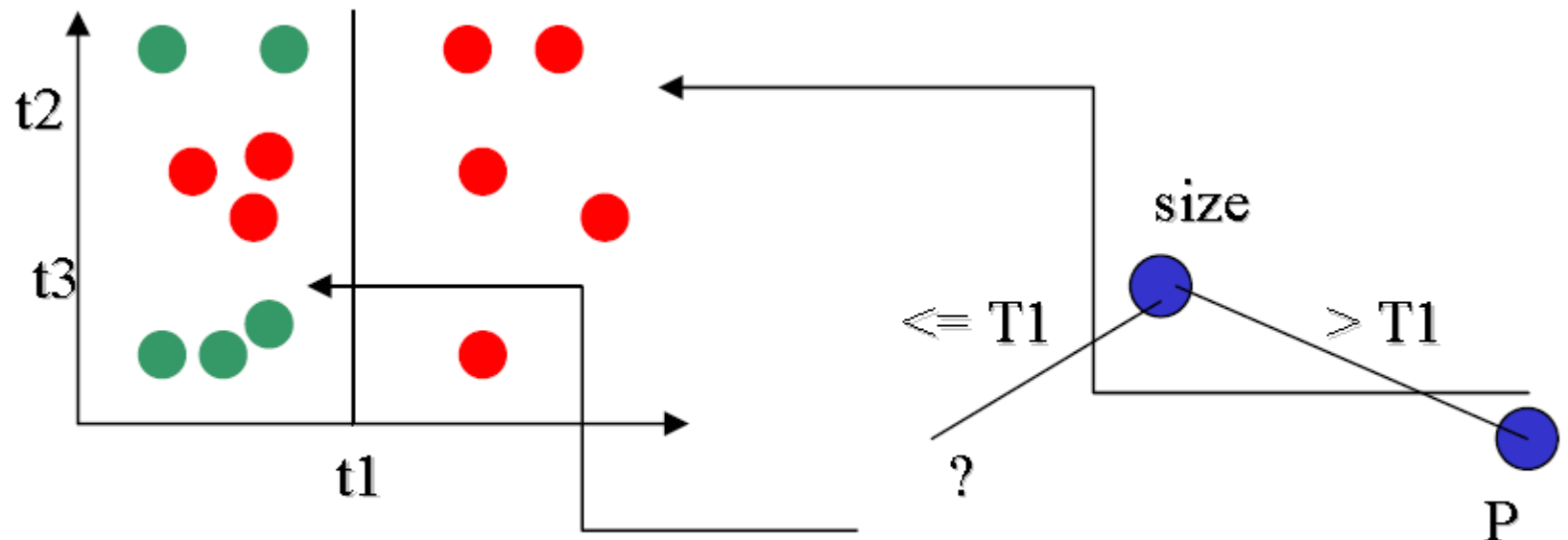
Attributes: size and humidity.

Size has two values: $>t1$ or $\leq t1$

Humidity has three values: $>t2$, ($>t3$ and $\leq t2$), $\leq t3$

Cont'd

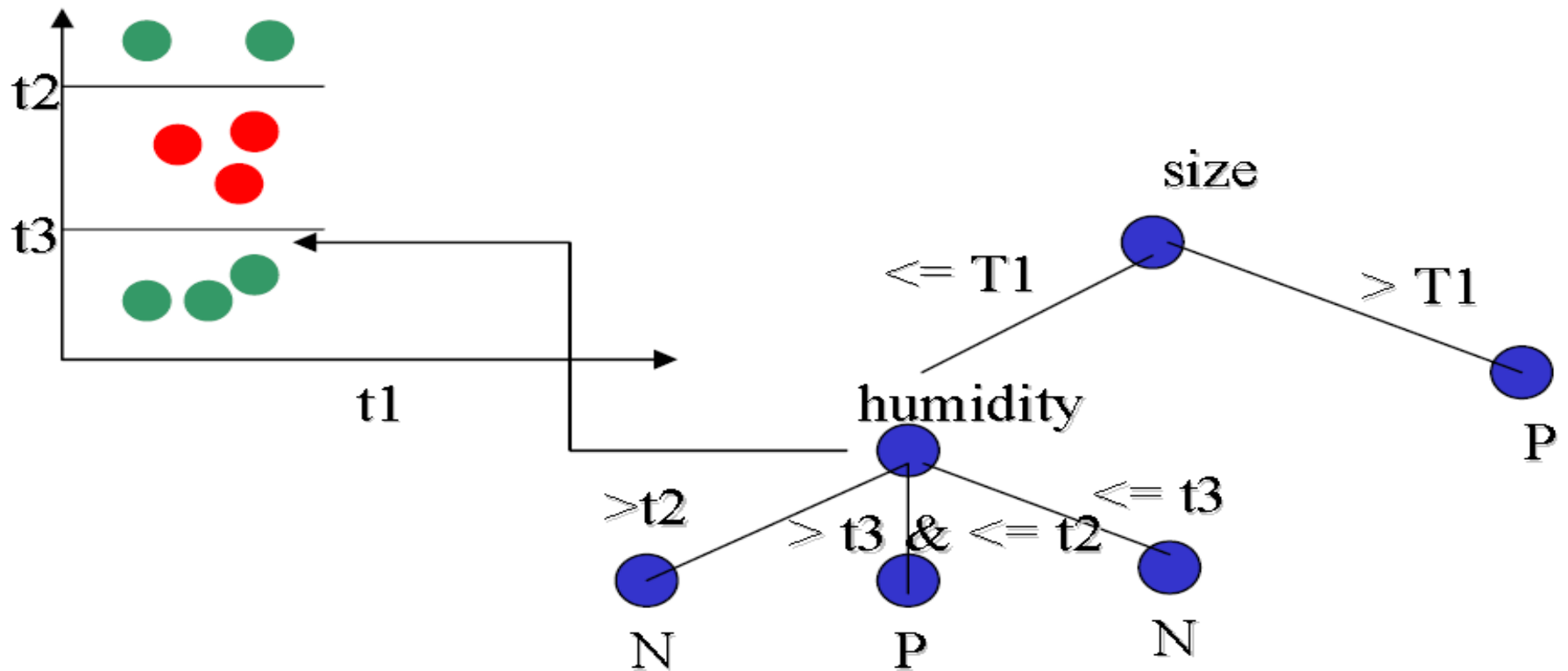
Suppose we choose **size** as the best



- Class P: poisonous
- Class N: not-poisonous

Cont'd

Suppose we choose **humidity** as the next best



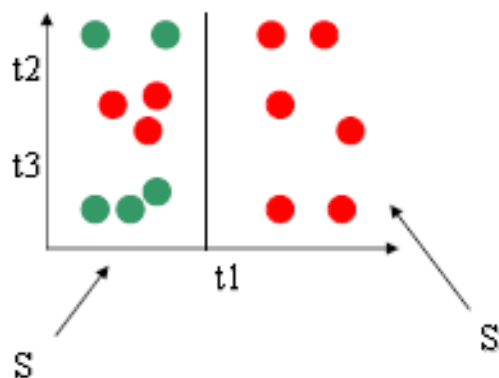
Cont'd

- ▶ **Steps:**
- ▶ • Create a root for the tree
- ▶ • If all examples are of the same class or the number of examples is below a threshold return that class
- ▶ • If no attributes available return majority class
- ▶ • Let a^* be the best attribute
- ▶ • For each possible value v of a^*
 - ▶ • Add a branch below a^* labeled “ $a = v$ ”
 - ▶ • Let S_v be the subsets of example where attribute $a^*=v$
 - ▶ • Recursively apply the algorithm to S_v

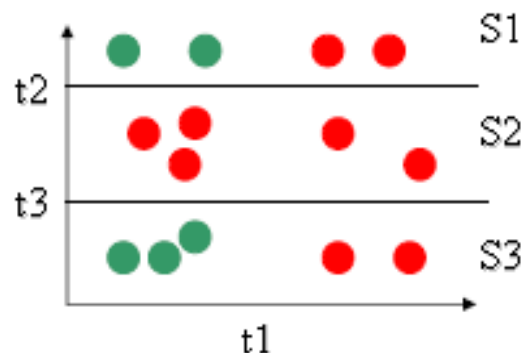
Rule Induction and Decision Tree - II

- ▶ **Splitting Functions**
- ▶ What attribute is the best to split the data? Let us remember some definitions from information theory.
- ▶ A measure of uncertainty or entropy that is associated to a random variable X is defined as
- ▶ $H(X) = - \sum p_i \log p_i$
- ▶ where the logarithm is in base 2.
- ▶ This is the “average amount of information or entropy of a finite complete probability scheme”.
- ▶ We will use a entropy based splitting function.
- ▶ Consider the previous example:

Cont'd



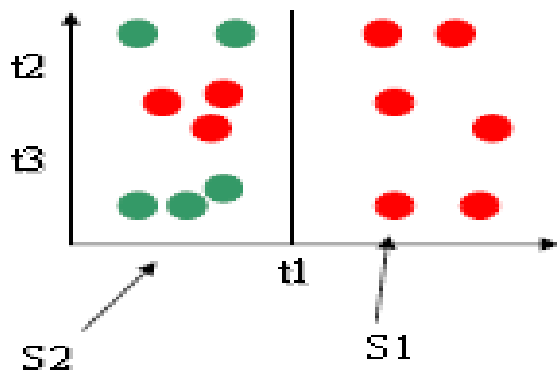
- ▶ Size divides the sample in two.
- ▶ $S1 = \{ 6P, 0NP \}$
- ▶ $S2 = \{ 3P, 5NP \}$
- ▶ $H(S1) = 0$
- ▶ $H(S2) = -(3/8)\log_2(3/8)$
- ▶ $-(5/8)\log_2(5/8)$



- ▶ humidity divides the sample in three.
- ▶ $S1 = \{ 2P, 2NP \}$
- ▶ $S2 = \{ 5P, 0NP \}$
- ▶ $S3 = \{ 2P, 3NP \}$
- ▶ $H(S1) = 1$
- ▶ $H(S2) = 0$
- ▶ $H(S3) = -(2/5)\log_2(2/5)$
- ▶ $-(3/5)\log_2(3/5)$

Cont'd

- ▶ Let us define *information gain* as follows:
- ▶ Information gain IG over attribute A: $IG(A)$
- ▶ $IG(A) = H(S) - \sum_v (S_v/S) H(S_v)$
- ▶ $H(S)$ is the entropy of all examples. $H(S_v)$ is the entropy of one subsample after partitioning S based on all possible values of attribute A.
- ▶ Consider the previous example:



We have,

$$H(S1) = 0$$

$$H(S2) = -(3/8)\log_2(3/8)$$

$$-(5/8)\log_2(5/8)$$

$$H(S) = -(9/14)\log_2(9/14)$$

$$-(5/14)\log_2(5/14)$$

$$|S1|/|S| = 6/14$$

$$|S2|/|S| = 8/14$$

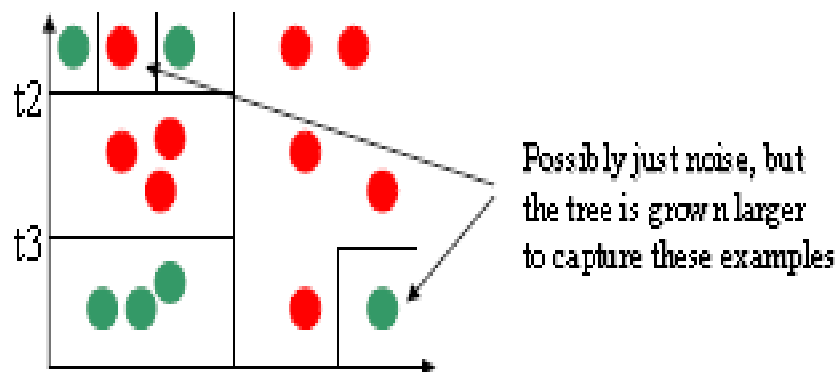
The principle for decision tree construction may be stated as follows:

Order the splits (attribute and value of the attribute) in decreasing order of information gain.

Decision Tree Pruning

- ▶ Practical issues while building a decision tree can be enumerated as follows:
 - ▶ 1) How deep should the tree be?
 - ▶ 2) How do we handle continuous attributes?
 - ▶ 3) What is a good splitting function?
 - ▶ 4) What happens when attribute values are missing?
 - ▶ 5) How do we improve the computational efficiency
- ▶ The depth of the tree is related to the generalization capability of the tree. If not carefully chosen it may lead to overfitting.
- ▶ A tree *overfits* the data if we let it grow deep enough so that it begins to capture “aberrations” in the data that harm the predictive power on unseen examples:

Cont'd



- ▶ There are two main solutions to overfitting in a decision tree:
- ▶ 1) stop the tree early before it begins to overfit the data
- ▶ + In practice this solution is hard to implement because I stopping point.
- ▶ 2) Grow the tree until the algorithm stops even if the overfitting problem shows up.
- ▶ Then prune the tree as a post-processing step.
- ▶ + This method has found great popularity in the machine learning community.

Cont'd

A common *decision tree pruning algorithm* is described below.

1. Consider all internal nodes in the tree.
2. For each node check if removing it (along with the subtree below it) and assigning the most common class to it does not harm accuracy on the validation set.
3. Pick the node n^* that yields the best performance and prune its subtree.
4. Go back to (2) until no more improvements are possible.

Decision trees are appropriate for problems where:

- Attributes are both numeric and nominal.
- Target function takes on a discrete number of values.
- A DNF representation is effective in representing the target concept.
- Data may have errors.

Learning and Neural Networks - I

- ▶ **Neural Networks**
- ▶ Artificial neural networks are among the most powerful learning models.
- ▶ They have the versatility to approximate a wide range of complex functions representing multi-dimensional input-output maps.
- ▶ Neural networks also have inherent adaptability, and can perform robustly even in noisy environments.
- ▶ An Artificial Neural Network (ANN) is an information processing paradigm that is inspired by the way biological nervous systems, such as the brain, process information.
- ▶ The key element of this paradigm is the novel structure of the information processing system.
- ▶ It is composed of a large number of highly interconnected simple processing elements (neurons) working in unison to solve specific problems.

Cont'd

- ▶ ANNs, like people, learn by example. An ANN is configured for a specific application, such as pattern recognition or data classification, through a learning process.
- ▶ Learning in biological systems involves adjustments to the synaptic connections that exist between the neurons. This is true of ANNs as well.
- ▶ ANNs can process information at a great speed owing to their highly massive parallelism.
- ▶ Neural networks, with their remarkable ability to derive meaning from complicated or imprecise data, can be used to extract patterns and detect trends that are too complex to be noticed by either humans or other computer techniques.

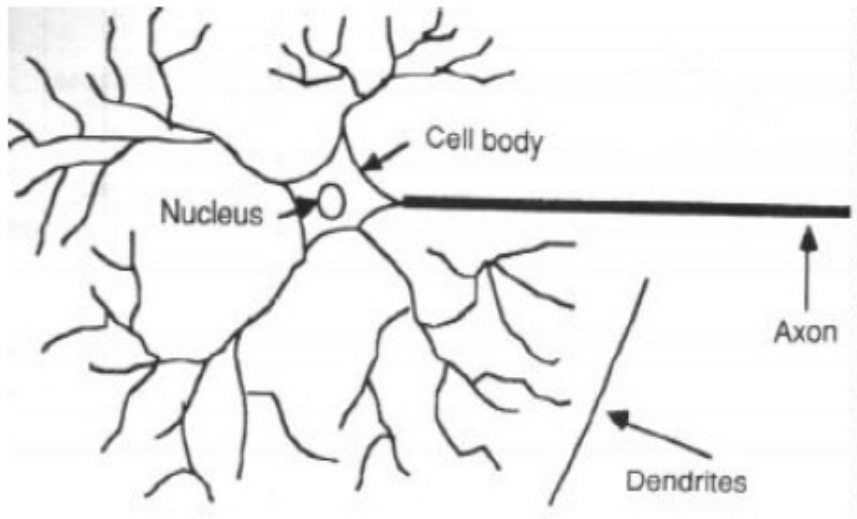
Cont'd

- ▶ A trained neural network can be thought of as an "expert" in the category of information it has been given to analyse.
- ▶ This expert can then be used to provide projections given new situations of interest and answer "what if" questions. Other advantages include:
 - ▶ 1. Adaptive learning: An ability to learn how to do tasks based on the data given for training or initial experience.
 - ▶ 2. Self-Organisation: An ANN can create its own organisation or representation of the information it receives during learning time.
 - ▶ 3. Real Time Operation: ANN computations may be carried out in parallel, and special hardware devices are being designed and manufactured which take advantage of this capability.
 - ▶ 4. Fault Tolerance via Redundant Information Coding: Partial destruction of a network leads to the corresponding degradation of performance. However, some network capabilities may be retained even with major network damage.

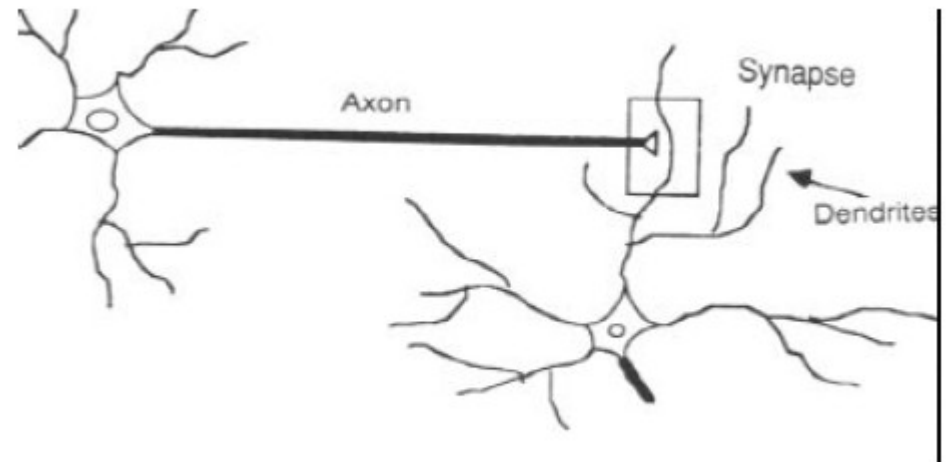
Biological Neural Networks

- ▶ Much is still unknown about how the brain trains itself to process information, so theories abound. In the human brain, a typical neuron collects signals from others through a host of fine structures called *dendrites*.
- ▶ The neuron sends out spikes of electrical activity through a long, thin strand known as an *axon*, which splits into thousands of branches.
- ▶ At the end of each branch, a structure called a *synapse* converts the activity from the axon into electrical effects that inhibit or excite activity from the axon into electrical effects that inhibit or excite activity in the connected neurones.
- ▶ When a neuron receives excitatory input that is sufficiently large compared with its inhibitory input, it sends a spike of electrical activity down its axon.
- ▶ Learning occurs by changing the effectiveness of the synapses so that the influence of one neuron on another changes.

Cont'd



Components of a Biological Neuron



The Synapse

Artificial Neural Networks

- ▶ Artificial neural networks are represented by a set of nodes, often arranged in layers, and a set of weighted directed links connecting them. The nodes are equivalent to neurons, while the links denote synapses. The nodes are the information processing units and the links acts as communicating media.
- ▶ There are a wide variety of networks depending on the nature of information processing carried out at individual nodes, the topology of the links, and the algorithm for adaptation of link weights. Some of the popular among them include:
- ▶ **Perceptron:** This consists of a single neuron with multiple inputs and a single output. It has restricted information processing capability. The information processing is done through a transfer function which is either linear or non-linear.

Cont'd

- ▶ **Multi-layered Perceptron (MLP):** It has a layered architecture consisting of input, hidden and output layers. Each layer consists of a number of perceptrons. The output of each layer is transmitted to the input of nodes in other layers through weighted links. Usually, this transmission is done only to nodes of the next layer, leading to what are known as feed forward networks. MLPs were proposed to extend the limited information processing capabilities of simple perceptrons, and are highly versatile in terms of their approximation ability. Training or weight adaptation is done in MLPs using supervised backpropagation learning.
- ▶ **Recurrent Neural Networks:** RNN topology involves backward links from output to the input and hidden layers. The notion of time is encoded in the RNN information processing scheme. They are thus used in applications like speech processing where inputs are time sequences data.

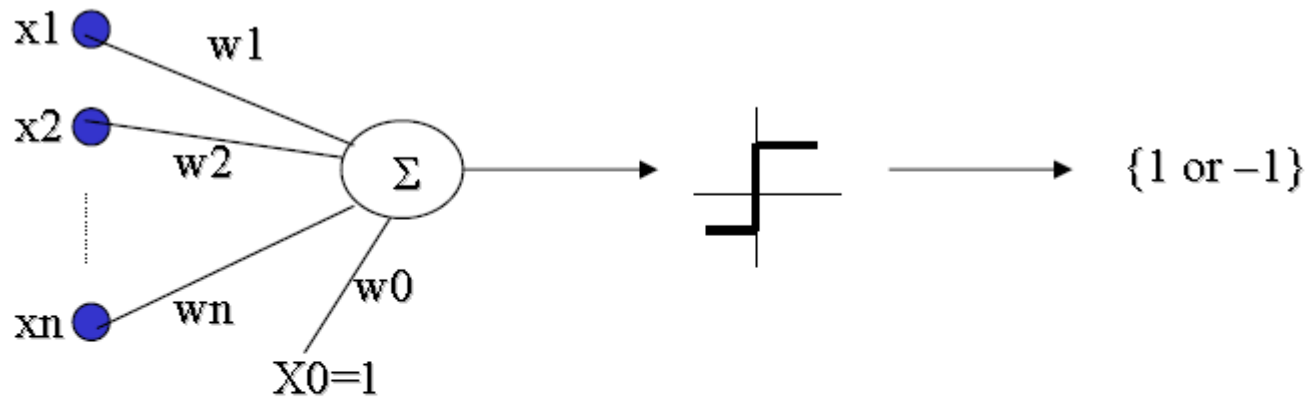
Cont'd

- ▶ **Self-Organizing Maps:** SOMs or Kohonen networks have a grid topology, with unequal grid weights. The topology of the grid provides a low dimensional visualization of the data distribution. These are thus used in applications which typically involve organization and human browsing of a large volume of data. Learning is performed using a winner take all strategy in a unsupervised mode.

Neural Networks - II

▶ Perceptron

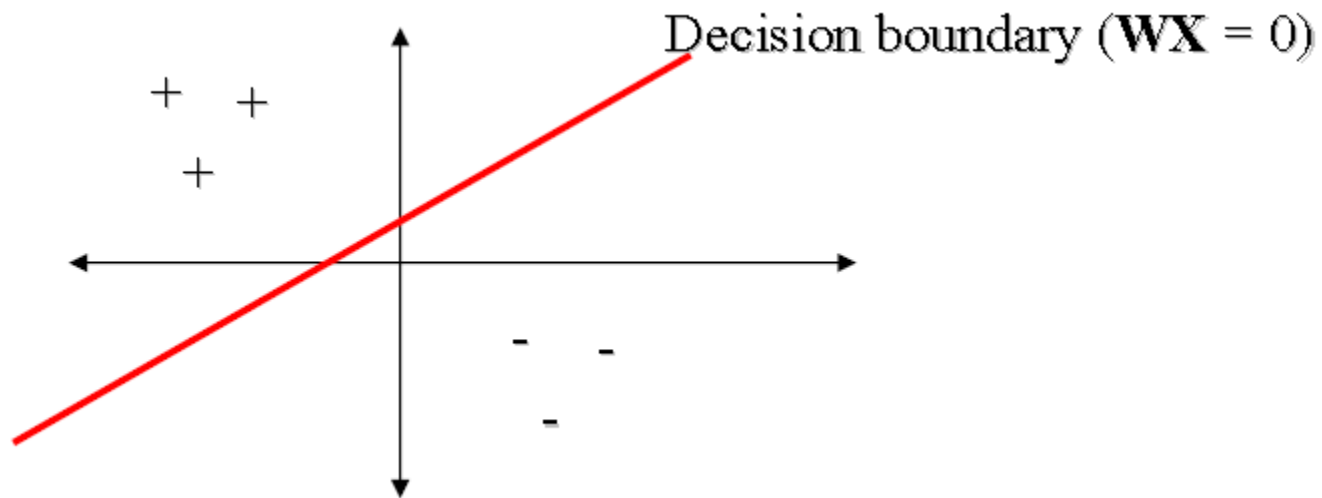
- ▶ *Definition:* It's a step function based on a linear combination of real-valued inputs. If the combination is above a threshold it outputs a 1, otherwise it outputs a -1 .



$$O(x_1, x_2, \dots, x_n) = \begin{cases} 1 & \text{if } w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n > 0 \\ -1 & \text{otherwise} \end{cases}$$

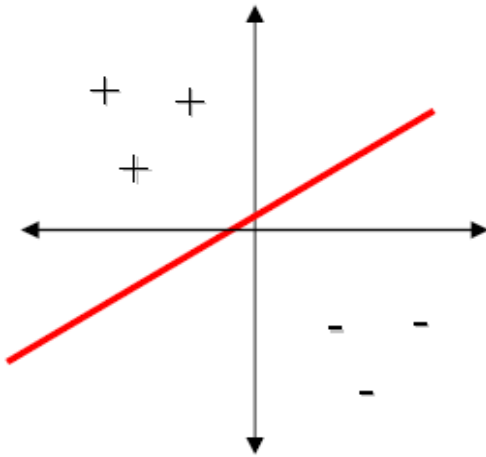
Cont'd

- ▶ A perceptron draws a hyperplane as the decision boundary over the (n-dimensional) input space.

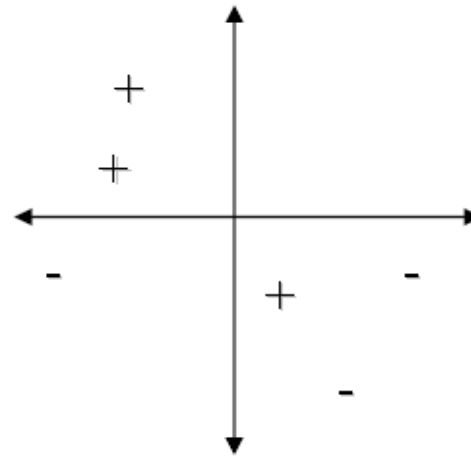


- A perceptron can learn only examples that are called “linearly separable”.
- These are examples that can be perfectly separated by a hyperplane.

Cont'd



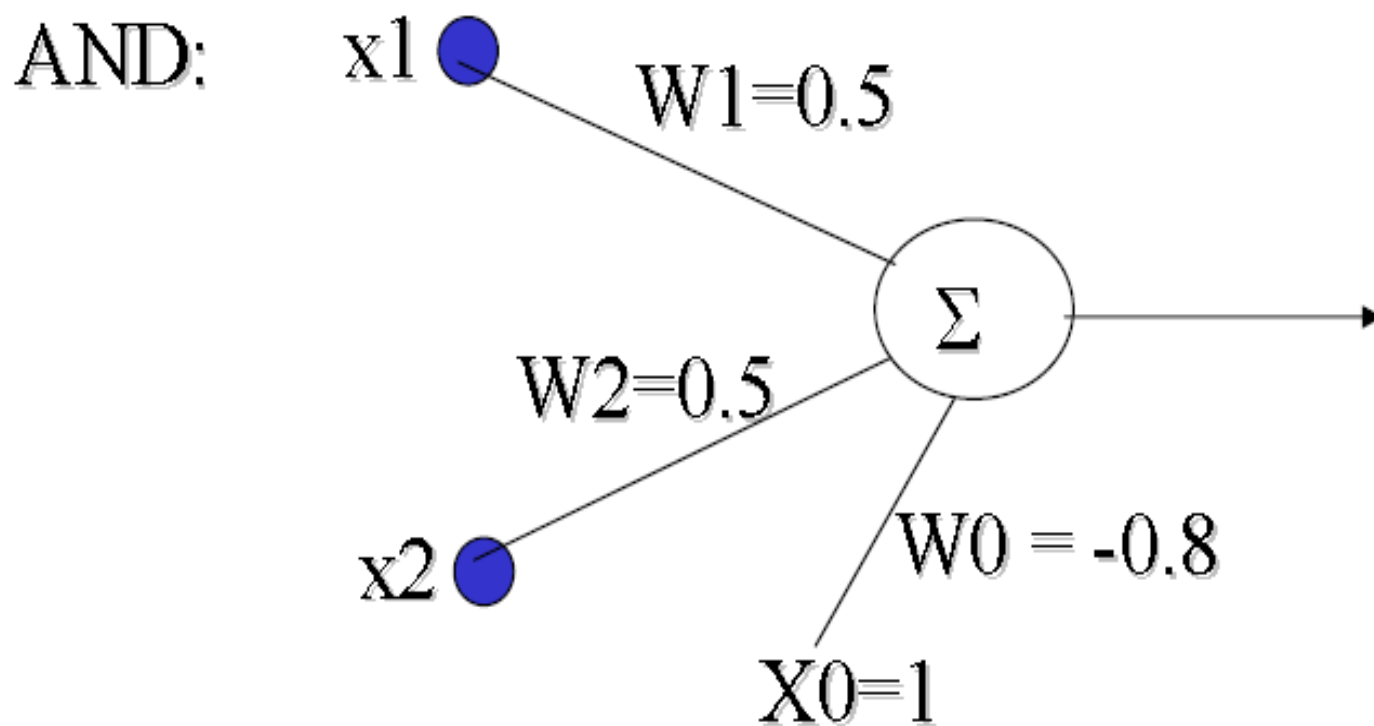
Linearly separable



Non-linearly separable

- ▶ Perceptrons can learn many boolean functions: AND, OR, NAND, NOR, but not XOR
- ▶ However, every boolean function can be represented with a perceptron network that has two levels of depth or more.
- ▶ The weights of a perceptron implementing the AND function is shown below.

Cont'd



Perceptron Learning

- ▶ Learning a perceptron means finding the right values for W .
- ▶ The hypothesis space of a perceptron is the space of all weight vectors.
- ▶ The perceptron learning algorithm can be stated as below.
 - ▶ 1. Assign random values to the weight vector
 - ▶ 2. Apply the *weight update rule* to every training example
 - ▶ 3. Are all training examples correctly classified?
 - ▶ a. Yes. Quit
 - ▶ b. No. Go back to Step 2.
- ▶ There are two popular weight update rules.
 - ▶ i) The perceptron rule, and
 - ▶ ii) Delta rule

The Perceptron Rule

- ▶ For a new training example $X = (x_1, x_2, \dots, x_n)$, update each weight according to this rule:
- ▶ $w_i = w_i + \Delta w_i$
- ▶ Where $\Delta w_i = \eta (t - o) x_i$
 - ▶ t : target output
 - ▶ o : output generated by the perceptron
 - ▶ η : constant called the learning rate (e.g., 0.1)
- ▶ Comments about the perceptron training rule:
 - ▶ If the example is correctly classified the term $(t - o)$ equals zero, and no update on the weight is necessary.
 - ▶ If the perceptron outputs -1 and the real answer is 1 , the weight is increased.
 - ▶ If the perceptron outputs a 1 and the real answer is -1 , the weight is decreased.
 - ▶ Provided the examples are linearly separable and a small value for η is used, the rule is proved to classify all training examples correctly (i.e, is consistent with the training data).

The Delta Rule

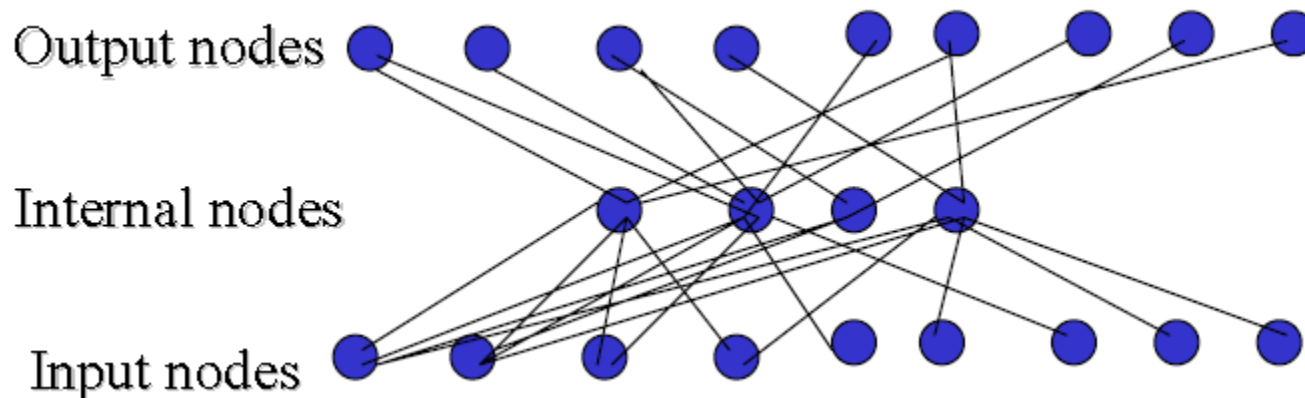
- ▶ What happens if the examples are not linearly separable?
- ▶ To address this situation we try to approximate the real concept using the delta rule.
- ▶ The key idea is to use a *gradient descent search*. We will try to minimize the following error:
- ▶ $E = \frac{1}{2} \sum_i (t_i - o_i)^2$
- ▶ where the sum goes over all training examples. Here o_i is the inner product WX and not $\text{sgn}(WX)$ as with the perceptron rule. The idea is to find a minimum in the space of weights and the error function E .
- ▶ The delta rule is as follows:
- ▶ For a new training example $X = (x_1, x_2, \dots, x_n)$, update each weight according to this rule:
- ▶ $w_i = w_i + \Delta w_i$
- ▶ Where $\Delta w_i = -\eta E'(\mathbf{W})/w_i$
- ▶ η : learning rate (e.g., 0.1)

Cont'd

- ▶ It is easy to see that
- ▶ $E'(W)/w_i = \sum_j (t_j - o_j) (-x_{ji})$
- ▶ So that gives us the following equation:
- ▶ $w_i = \eta \sum_j (t_j - o_j) x_{ji}$
- ▶ There are two differences between the perceptron and the delta rule. The perceptron is based on an output from a step function, whereas the delta rule uses the linear combination of inputs directly. The perceptron is guaranteed to converge to a consistent hypothesis assuming the data is linearly separable. The delta rule converges in the limit but it does not need the condition of linearly separable data.
- ▶ There are two main difficulties with the gradient descent method:
 - ▶ 1. Convergence to a minimum may take a long time.
 - ▶ 2. There is no guarantee we will find the global minimum.
- ▶ These are handled by using momentum terms and random perturbations to the weight vectors.

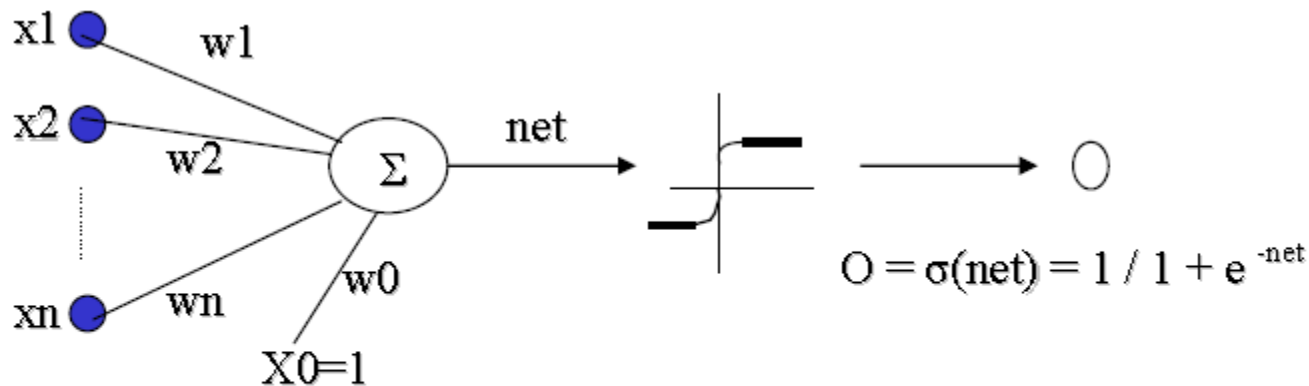
Neural Networks - III

- ▶ Multi-Layer Perceptrons
- ▶ In contrast to perceptrons, multilayer networks can learn not only multiple decision boundaries, but the boundaries may be nonlinear.
- ▶ The typical architecture of a multi-layer perceptron (MLP) is shown below.



- To make nonlinear partitions on the space we need to define each unit as a nonlinear function (unlike the perceptron). One solution is to use the sigmoid unit.
- Another reason for using sigmoids are that they are continuous unlike linear thresholds and are thus differentiable at all points.

Cont'd



$$O(x_1, x_2, \dots, x_n) = \sigma(WX)$$

$$\text{where: } \sigma(WX) = 1 / 1 + e^{-WX}$$

Function σ is called the sigmoid or logistic function. It has the following property:

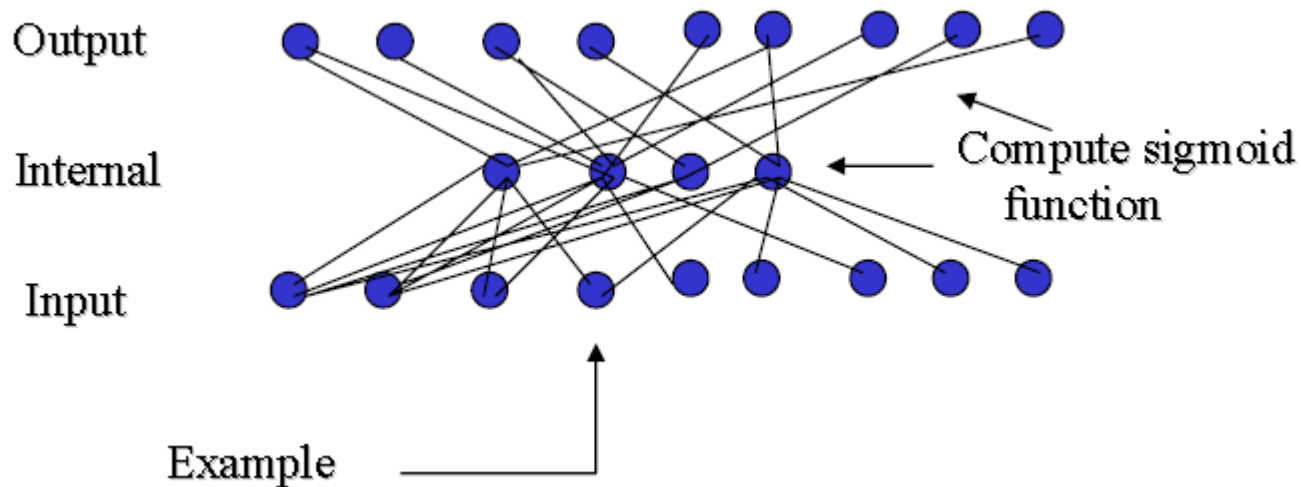
$$d \sigma(y) / dy = \sigma(y) (1 - \sigma(y))$$

Back-Propagation Algorithm

- ▶ Multi-layered perceptrons can be trained using the back-propagation algorithm described next.
- ▶ Goal: To learn the weights for all links in an interconnected multilayer network.
- ▶ We begin by defining our measure of error:
- ▶ $E(W) = \frac{1}{2} \sum_d \sum_k (t_{kd} - o_{kd})^2$
- ▶ k varies along the output nodes and d over the training examples.
- ▶ The idea is to use again a gradient descent over the space of weights to find a global minimum (no guarantee).
- ▶ **Algorithm:**
 - ▶ 1. Create a network with n_{in} input nodes, n_{hidden} internal nodes, and n_{out} output nodes.
 - ▶ 2. Initialize all weights to small random numbers.
 - ▶ 3. Until error is small do:
 - ▶ For each example X do
 - ▶ • Propagate example X forward through the network
 - ▶ • Propagate errors backward through the network

Cont'd

- ▶ Forward Propagation
- ▶ Given example X , compute the output of every node until we reach the output nodes:



Backward Propagation

- ▶ A. For each output node k compute the error:

$$\delta_k = O_k (1 - O_k)(t_k - O_k)$$

- ▶ B. For each hidden unit h , calculate the error:

$$\delta_h = O_h (1 - O_h) \sum_k W_{kh} \delta_k$$

- ▶ C. Update each network weight:

$$C. W_{ji} = W_{ji} + \Delta W_{ji}$$

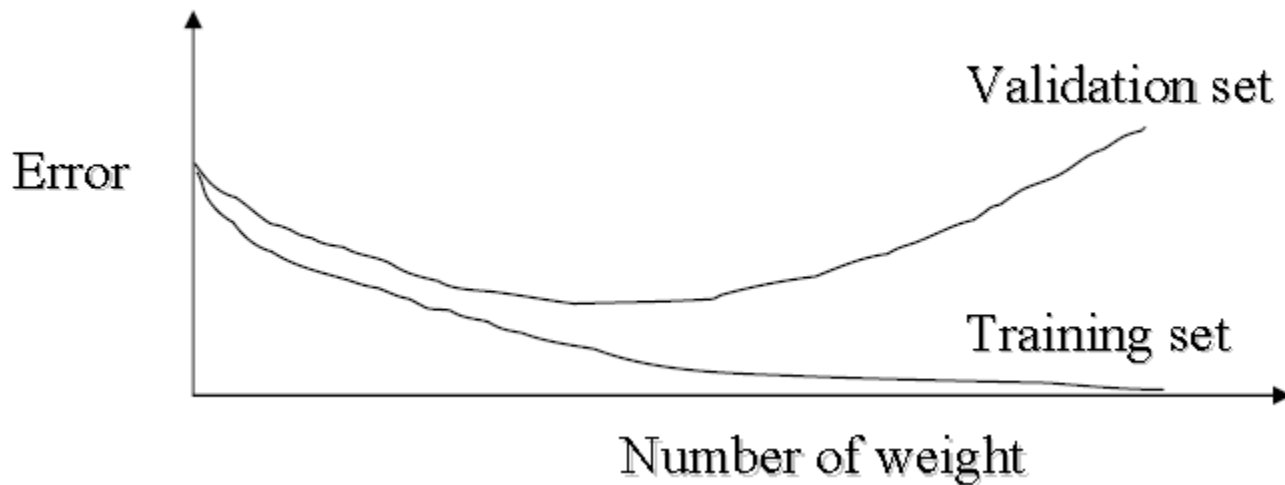
- ▶ where $\Delta W_{ji} = \eta \delta_j X_{ji}$ (W_{ji} and X_{ji} are the input and weight of node i to node j)
- ▶ A momentum term, depending on the weight value at last iteration, may also be added to the update rule as follows. At iteration n we have the following:
- ▶ $\Delta W_{ji}(n) = \eta \delta_j X_{ji} + \alpha \Delta W_{ji}(n)$
- ▶ Where α ($0 \leq \alpha \leq 1$) is a constant called the momentum.
 - ▶ 1. It increases the speed along a local minimum.
 - ▶ 2. It increases the speed along flat regions.

Cont'd

- ▶ *Remarks on Back-propagation*
- ▶ 1. It implements a gradient descent search over the weight space.
- ▶ 2. It may become trapped in local minima.
- ▶ 3. In practice, it is very effective.
- ▶ 4. How to avoid local minima?
 - ▶ a) Add momentum.
 - ▶ b) Use stochastic gradient descent.
 - ▶ c) Use different networks with different initial values for the weights.
- ▶ Multi-layered perceptrons have high representational power. They can represent the following:
 - ▶ 1. Boolean functions. Every boolean function can be represented with a network having two layers of units.
 - ▶ 2. Continuous functions. All bounded continuous functions can also be approximated with a network having two layers of units.
 - ▶ 3. Arbitrary functions. Any arbitrary function can be approximated with a network with three layers of units.

Generalization and overfitting

- ▶ One obvious stopping point for backpropagation is to continue iterating until the error is below some threshold; this can lead to overfitting.



Overfitting can be avoided using the following strategies.

- Use a validation set and stop until the error is small in this set.
- Use 10 fold cross validation.
- Use weight decay; the weights are decreased slowly on each iteration.

Applications of Neural Networks

- ▶ Neural networks have broad applicability to real world business problems. They have already been successfully applied in many industries.
- ▶ Since neural networks are best at identifying patterns or trends in data, they are well suited for prediction or forecasting needs including:
 - ▶ • sales forecasting
 - ▶ • industrial process control
 - ▶ • customer research
 - ▶ • data validation
 - ▶ • risk management
 - ▶ • target marketing
- ▶ Because of their adaptive and non-linear nature they have also been used in a number of control system application domains like, • process control in chemical plants • unmanned vehicles • robotics • consumer electronics
- ▶ Neural networks are also used a number of other applications which are too hard to model using classical techniques. These include, computer vision, path planning and user modeling.